

Jobsheet 12 — Integrasi Modul Peminjaman

Sub-CPMK: Mengintegrasikan front-end dan back-end proyek secara utuh.

Perubahan dari Jobsheet 11

- Tambah `sql/03_peminjaman.sql` — tabel `peminjaman` (relasi ke `buku` dan `anggota`), melengkapi ERD yang sudah dirancang di Jobsheet 8.
- Tambah modul **Peminjaman** (menghubungkan seluruh entitas yang sudah dibangun sejak Jobsheet 8-10 sekaligus):
 - `peminjaman/tambah.php` + `proses_tambah.php`: pilih anggota + buku (dropdown hanya `stok > 0`), simpan transaksi **dan** kurangi stok buku dalam satu **transaction** (`beginTransaction / commit / rollBack`) dengan `SELECT ... FOR UPDATE` untuk mencegah race condition stok.
 - `peminjaman/kembali.php` + `proses_kembali.php`: daftar transaksi aktif (`status = 'dipinjam'`), tombol Kembalikan menambah kembali stok buku dalam transaction serupa.
 - `peminjaman/riwayat.php`: histori peminjaman per anggota (`JOIN peminjaman + buku`).
- `includes/header.php`: navbar menambahkan menu Peminjaman Baru, Pengembalian, Riwayat (hanya saat login).
- `index.php`: kartu "Sedang Dipinjam" kini `COUNT(*) FROM peminjaman WHERE status = 'dipinjam'` (sebelumnya statis `0`).

Cara menjalankan

```
psql -d simpus_mini -f sql/03_peminjaman.sql
```

Opsi 1 — PHP built-in server:

```
php -S localhost:8000
```

Opsi 2 — Laragon (Apache): lewat virtual host langsung ke folder `jobsheet-12/` (mis. `http://jobsheet12.test/`), atau bersarang di bawah domain proyek (mis. `http://dp2026.test/kode-praktikum/jobsheet-12/`) — path CSS/JS/link/redirect login sudah relatif otomatis (lihat `includes/header.php` & `includes/auth.php`), jadi keduanya jalan.

Pengujian end-to-end yang disarankan

Registrasi petugas → Login → Tambah Buku & Anggota → Peminjaman Baru → cek stok buku berkurang di Daftar Buku → cek kartu "Sedang Dipinjam" di Beranda bertambah → Pengembalian → cek stok kembali bertambah dan transaksi hilang dari daftar aktif → Riwayat (pilih anggota) → transaksi muncul berstatus "Selesai" → Logout.

Catatan

- Validasi bisnis tambahan (anggota dengan peminjaman terlambat >14 hari tidak boleh meminjam buku baru) belum diterapkan — jadi tugas mandiri.
- Operasi stok memakai `SELECT ... FOR UPDATE` di dalam transaction, bukan sekadar `UPDATE` buku `SET stok = stok - 1` tanpa pengecekan, agar stok tidak bisa menjadi negatif bila dua peminjaman diproses hampir bersamaan.

Wireframe & User Flow — SIMPUS-Mini

Sub-CPMK: Merancang UI/UX aplikasi (proyek).

Halaman yang sudah ada (Beranda, Daftar/Tambah Buku, Daftar/Tambah Anggota — Jobsheet 1-3) belum mencakup fitur Login, Dashboard Petugas, dan Peminjaman/Pengembalian. Dokumen ini merancang wireframe untuk halaman-halaman tersebut sebelum diimplementasikan mulai Jobsheet 5 dan seterusnya.

Aktor

- **Tamu:** hanya bisa melihat katalog buku (Beranda, Daftar Buku) tanpa login.
- **Petugas:** login untuk mengakses seluruh fitur CRUD dan transaksi peminjaman.

User Flow — Peminjaman Buku

```
[Petugas Login] -> [Dashboard] -> [Pilih menu "Peminjaman Baru"]
-> [Pilih Anggota] -> [Pilih Buku (stok > 0)]
-> [Simpan] -> [Stok buku berkurang 1] -> [Kembali ke Dashboard]
```

User Flow — Pengembalian Buku

```
[Dashboard] -> [Menu "Pengembalian"] -> [Cari transaksi aktif (anggota/buku)]  
-> [Tandai "Dikembalikan"] -> [Stok buku bertambah 1]  
-> [Kembali ke Dashboard]
```

Wireframe: Halaman Login

```
+-----+  
| SIMPUS-Mini |  
+-----+  
| [ Login Petugas ] |  
| |  
| Username : [ _____ ] |  
| Password : [ _____ ] |  
| |  
| [ Masuk ] |  
| |  
| Belum punya akun? Daftar di sini |  
+-----+
```

Wireframe: Dashboard Petugas

```
+-----+  
| SIMPUS-Mini Beranda | Buku | Anggota | Peminjaman | (Nama Petugas) Logout |  
+-----+  
| [Total Buku] [Total Anggota] [Sedang Dipinjam] |  
| |  
| Aksi Cepat: |  
| [ + Peminjaman Baru ] [ + Pengembalian ] |  
| |  
| Transaksi Terbaru |  
| ----- |  
| Anggota | Buku | Tgl Pinjam | Status |  
+-----+
```

Wireframe: Form Peminjaman

```
+-----+
|  Form Peminjaman Buku  |
|-----|
|  Anggota : [ dropdown pilih anggota ] |
|  Buku    : [ dropdown, hanya stok>0 ] |
|  Tanggal Pinjam : [ auto: hari ini ] |
|                                     |
|          [ Simpan Peminjaman ]      |
+-----+
```

Wireframe: Form Pengembalian

```
+-----+
|  Pengembalian Buku  |
|-----|
|  Cari transaksi aktif: |
|  [ nama anggota / judul buku _____ ] |
|                                     |
|  Anggota | Buku | Tgl Pinjam | [Kembalikan] |
+-----+
```

Wireframe: Riwayat Peminjaman per Anggota

```
+-----+
|  Riwayat Peminjaman – Siti Aminah  |
|-----|
|  Buku      | Pinjam  | Kembali | Status      |
|  Laskar Pelangi  | 01/07  | 10/07  | Selesai      |
|  Bumi Manusia   | 15/07  | -      | Dipinjam      |
+-----+
```

Konsistensi dengan Desain yang Sudah Berjalan

- Warna aksen, tipografi navbar, dan gaya tabel/kartu mengikuti `assets/css/style.css` yang sudah dibangun sejak Jobsheet 2-3.
- Navbar akan ditambah menu **Peminjaman** dan indikator status login (nama petugas / tombol Logout) mulai implementasi di Jobsheet 10.
- Edge case yang perlu ditangani saat implementasi: buku stok habis tidak boleh dipilih di form peminjaman; anggota dengan tunggakan terlambat divalidasi di Jobsheet 12 (tugas mandiri).

Dokumentasi Jobsheet 12 – Integrasi Modul Peminjaman

Dokumentasi ini melanjutkan [dokumentasi jobsheet-11](#) (Keamanan Web Dasar). Jobsheet-12 adalah **puncak** dari seluruh rangkaian jobsheet SIMPUS-Mini – ingat wireframe Peminjaman/Pengembalian yang sudah kamu bedah sejak [dokumentasi jobsheet-04 §3](#), 8 jobsheet yang lalu – sekarang **benar-benar dibangun**, menghubungkan **semua** yang sudah kamu pelajari: HTML/CSS, PHP, database, sesi, otorisasi, dan keamanan, sekaligus dalam satu fitur.

Tentang `docs/wireframe.md` **dan** `docs/security-checklist.md`

Kedua file ini **identik persis** dengan versi di [jobsheet-11](#). Tidak ada rancangan UI/UX maupun audit keamanan baru – jobsheet ini murni **mengimplementasikan** apa yang sudah dirancang dan diamankan sebelumnya.

Apa yang Baru di Jobsheet 12?

Sesuai [README.md](#) jobsheet ini:

1. `sql/03_peminjaman.sql` – tabel `peminjaman` baru, **terhubung** (lewat *foreign key*) ke tabel `buku` dan `anggota` yang sudah ada sejak jobsheet-08.
2. `peminjaman/tambah.php` + `proses_tambah.php` – mencatat peminjaman baru **dan** mengurangi stok buku **dalam satu transaction****, dengan `SELECT ... FOR UPDATE` untuk mencegah *race* condition**.
3. `peminjaman/kembali.php` + `proses_kembali.php` – daftar transaksi aktif, tombol Kembalikan yang menambah kembali stok buku.
4. `peminjaman/riwayat.php` – histori peminjaman per anggota, memakai `JOIN` untuk menggabungkan data dari 3 tabel sekaligus.

5. **Navbar** menambahkan 3 menu baru (Peminjaman Baru, Pengembalian, Riwayat), hanya untuk Petugas yang login.
6. **Kartu "Sedang Dipinjam"** di Beranda kini benar-benar dihitung dari database, menggantikan angka statis 0 sejak jobsheet-01.

Daftar Isi

1. [Konsep Dasar: Relasi Tabel & Transaction](#)
2. [Skema peminjaman & Relasi Antar Tabel](#)
3. [Peminjaman Baru & Database Transaction](#)
4. [Pengembalian Buku](#)
5. [Riwayat Peminjaman & JOIN](#)
6. [Rangkuman & Latihan Lanjutan](#)

Struktur Folder

```
jobsheet-12/
├── index.php                # Kartu "Sedang Dipinjam" kini dari database
├── includes/
│   ├── header.php         # + 3 menu Peminjaman
│   └── ...                 # Tidak berubah lainnya
├── peminjaman/
│   ├── tambah.php, proses_tambah.php # BARU - transaction + FOR UPDATE
│   ├── kembali.php, proses_kembali.php # BARU - transaction serupa
│   └── riwayat.php        # BARU - JOIN 2 tabel
├── sql/
│   ├── 01_buku_anggota.sql
│   ├── 02_users.sql
│   └── 03_peminjaman.sql   # BARU - tabel peminjaman + foreign key
├── buku/, anggota/, auth/  # Tidak berubah dari jobsheet-11
├── docs/
│   ├── wireframe.md        # Identik dengan jobsheet-11
│   └── security-checklist.md # Identik dengan jobsheet-11
├── README.md
└── Dokumentasi/            # Folder dokumentasi ini
```

Catatan penting dari [README.md](#) jobsheet ini: validasi bisnis tambahan (anggota dengan peminjaman terlambat >14 hari tidak boleh meminjam buku baru) **belum diterapkan** — dijadikan

tugas mandiri (lihat [bab 6](#)).

1. Konsep Dasar: Relasi Tabel & Transaction

Dua konsep besar yang benar-benar baru di jobsheet ini: **tabel yang** saling terhubung** (relasi), dan **transaction** (kelompok perintah SQL** yang harus berhasil/gagal bersama-sama).

1.1 Kenapa Peminjaman Butuh Tabel Sendiri?

Ingat tabel `buku` dan `anggota` sejak [dokumentasi jobsheet-08 §2](#) — masing-masing berdiri sendiri, tidak saling "tahu" satu sama lain. Tapi satu **transaksi peminjaman** pada dasarnya menyatukan **dua** informasi sekaligus: *buku mana* yang dipinjam **dan** *anggota mana* yang meminjamnya — plus informasi baru yang tidak dimiliki keduanya: kapan dipinjam, kapan (atau belum) dikembalikan, dan statusnya. Tabel `peminjaman` yang baru inilah yang menyimpan **hubungan** antara sebuah buku dan seorang anggota, ditambah data spesifik transaksi itu sendiri.

1.2 Foreign Key: Menghubungkan Antar Tabel

```
buku_id INTEGER NOT NULL REFERENCES buku(id),  
anggota_id INTEGER NOT NULL REFERENCES anggota(id),
```

`REFERENCES buku(id)` adalah **foreign key** (kunci asing) — aturan yang memberi tahu database: "kolom `buku_id` di tabel ini **harus** berisi nilai yang benar-benar ada sebagai `id` di tabel `buku`." Ini konsep baru yang melengkapi **PRIMARY KEY** yang sudah kamu kenal sejak [dokumentasi jobsheet-08 §2.4](#). Manfaatnya: database **sendiri** yang menolak kalau ada percobaan mencatat peminjaman dengan `buku_id` yang **tidak pernah ada** di tabel `buku` — mencegah data yang tidak konsisten ("peminjaman buku hantu") tanpa perlu kode PHP tambahan untuk memeriksanya secara manual. Penjelasan detail seluruh skema ada di [bab 2](#).

1.3 Kenapa Butuh Transaction?

Perhatikan alur mencatat peminjaman baru ([bab 3](#)) sebenarnya melibatkan **dua** perintah SQL terpisah:

1. `INSERT` baris baru ke tabel `peminjaman`.
2. `UPDATE` mengurangi `stok` di tabel `buku`.

Bayangkan kalau perintah **pertama** berhasil, tapi entah kenapa (misalnya server tiba-tiba mati) perintah **kedua** gagal dijalankan — hasilnya: ada catatan peminjaman yang tersimpan, tapi stok buku **tidak** pernah berkurang**. Data jadi tidak konsisten — sistem "berbohong"*** tentang berapa banyak stok buku yang sebenarnya tersedia.

Transaction menyelesaikan masalah ini: mengelompokkan beberapa perintah SQL supaya **semuanya berhasil bersama-sama, atau semuanya** dibatalkan bersama-sama** — tidak ada kondisi "setengah berhasil."** Detail cara memakainya (`beginTransaction`, `commit`, `rollback`) dibahas di [bab 3 §3.4](#).

1.4 Apa itu Race Condition?

Race condition adalah masalah yang muncul ketika **dua proses** berjalan hampir bersamaan** dan saling mengganggu hasil satu sama lain.** Contoh konkret di aplikasi ini: bayangkan sebuah buku stoknya tinggal **1**, dan **dua** petugas (di komputer berbeda) **secara bersamaan** mencoba meminjamkan buku itu ke anggota berbeda. Kalau kedua proses sama-sama membaca "stok = 1" **sebelum** salah satu sempat mengurangnya, keduanya bisa **sama-sama** menganggap peminjaman itu sah — hasil akhirnya, stok buku bisa menjadi **-1** (angka yang tidak masuk akal), atau dua orang meminjam buku fisik yang sama yang sebenarnya hanya ada satu eksemplar.

Solusi untuk masalah ini — `SELECT ... FOR UPDATE` — dibahas detail di [bab 3 §3.5](#).

1.5 Peta Modul Peminjaman

```
peminjaman/  
├─ tambah.php + proses_tambah.php → mencatat peminjaman baru, kurangi stok  
├─ kembali.php + proses_kembali.php → menandai selesai, tambah kembali stok  
└─ riwayat.php → melihat histori per anggota
```

Ketiga halaman ini **sama-sama** butuh data dari **lebih dari satu** tabel (`peminjaman`, `buku`, `anggota`) — inilah kenapa relasi tabel (§1.2) dan `JOIN` (dibahas di [bab 5](#)) menjadi penting di jobsheet ini, berbeda dari jobsheet-jobsheet sebelumnya yang tiap halaman biasanya hanya bekerja dengan **satu** tabel saja.

Lanjut ke: [Skema peminjaman & Relasi Antar Tabel](#)

2. Skema peminjaman & Relasi Antar Tabel

2.1 Kode Lengkap sql/03_peminjaman.sql

```
-- Jobsheet 12: tabel peminjaman, menghubungkan buku, anggota, dan users
-- Jalankan: psql -d simpus_mini -f sql/03_peminjaman.sql
```

```
CREATE TABLE IF NOT EXISTS peminjaman (
  id SERIAL PRIMARY KEY,
  buku_id INTEGER NOT NULL REFERENCES buku(id),
  anggota_id INTEGER NOT NULL REFERENCES anggota(id),
  tanggal_pinjam DATE NOT NULL DEFAULT CURRENT_DATE,
  tanggal_kembali DATE,
  status VARCHAR(20) NOT NULL DEFAULT 'dipinjam'
);
```

Elemen `SERIAL PRIMARY KEY`, `NOT NULL`, `VARCHAR`, `DEFAULT` sudah kamu kuasai sejak [dokumentasi jobsheet-08 §2](#). Mari bedah bagian-bagian yang baru.

2.2 Kolom Foreign Key: `buku_id` dan `anggota_id`

```
buku_id INTEGER NOT NULL REFERENCES buku(id),
anggota_id INTEGER NOT NULL REFERENCES anggota(id),
```

Ingat konsep foreign key dari [bab 1 §1.2](#). Perhatikan pola penamaan kolomnya: `buku_id` — gabungan nama tabel yang dirujuk (`buku`) ditambah `_id`. Ini konvensi penamaan yang umum dipakai supaya sekilas membaca nama kolom saja sudah cukup untuk menebak tabel mana yang dirujuknya, tanpa perlu membuka skema tabel `peminjaman` secara lengkap. Sama halnya dengan `anggota_id`, merujuk ke `anggota(id)`.

2.3 Tipe Data `DATE` dan `DEFAULT CURRENT_DATE`

```
tanggal_pinjam DATE NOT NULL DEFAULT CURRENT_DATE,
tanggal_kembali DATE,
```

- `DATE` — tipe data baru, khusus menyimpan **tanggal** (tanpa jam/menit/detik) — cocok untuk kolom seperti ini yang hanya perlu tahu "tanggal berapa," bukan waktu presisi ke detik.
- `DEFAULT CURRENT_DATE` — ingat konsep `DEFAULT` dari [dokumentasi jobsheet-08 §2.4](#), di sini dipasangkan dengan `CURRENT_DATE` — fungsi bawaan PostgreSQL yang selalu bernilai "tanggal hari ini." Kalau `INSERT` tidak menyebutkan nilai untuk `tanggal_pinjam` secara eksplisit, PostgreSQL otomatis mengisinya dengan tanggal saat baris itu dibuat —

mewujudkan wireframe form Peminjaman yang sudah kamu bedah sejak [dokumentasi jobsheet-04 §2.3](#):

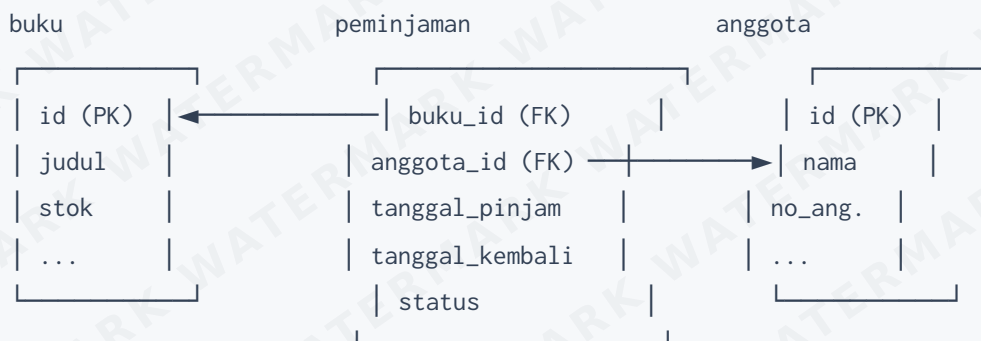
- "Tanggal Pinjam: [auto: hari ini]".
- tanggal_kembali DATE (tanpa NOT NULL) — kolom ini **boleh** kosong** (NULL), karena saat buku baru dipinjam, tanggal** kembalinya **belum diketahui** — kolom ini baru akan diisi saat proses Pengembalian ([bab 4](#)).

2.4 Kolom status

```
status VARCHAR(20) NOT NULL DEFAULT 'dipinjam'
```

Menyimpan **status** transaksi peminjaman, dengan **dua** nilai yang mungkin di aplikasi ini: 'dipinjam' (nilai default saat baru dibuat) atau 'dikembalikan' (diubah saat proses Pengembalian). Kolom ini persis mewujudkan kolom "Status" yang sudah kamu bedah di wireframe Riwayat Peminjaman sejak [dokumentasi jobsheet-04 §2.3](#).

2.5 Diagram Relasi Sederhana



- **PK (Primary Key)** — kunci utama tiap tabel, ingat dari [dokumentasi jobsheet-08 §2.4](#).
- **FK (Foreign Key)** — kolom yang **menunjuk** ke Primary Key tabel lain, ingat dari [§2.2](#).

Tabel `peminjaman` berada **di tengah**, secara harfiah menjadi "jembatan" yang menghubungkan satu baris `buku` dengan satu baris `anggota` — pola desain database yang sangat umum, disebut **relasi banyak-ke-banyak**** (*many-to-many*): satu buku bisa dipinjam oleh** banyak anggota berbeda **dari waktu ke waktu** (satu per satu, karena stoknya terbatas), dan satu anggota bisa meminjam banyak buku berbeda.

2.6 Menjalankan Skema Tambahan

Sesuai [README.md](#) jobsheet ini:

```
psql -d simpus_mini -f sql/03_peminjaman.sql
```

Ingat pola perintah ini dari [dokumentasi jobsheet-08 §3.3](#) — menjalankan file skema tambahan **tanpa** mengulang seluruh setup dari awal, karena `CREATE TABLE IF NOT EXISTS` ([dokumentasi jobsheet-08 §2.3](#)) aman dijalankan kapan pun tanpa mengganggu tabel `buku / anggota / users` yang sudah ada.

Lanjut ke: [Peminjaman Baru & Database Transaction](#)

3. Peminjaman Baru & Database Transaction

Bab paling teknis di jobsheet ini — menggabungkan transaction database dengan penguncian baris (`FOR UPDATE`) untuk mencegah race condition.

3.1 peminjaman/tambah.php: Menyiapkan Dropdown

```
$daftarAnggota = $pdo->query("SELECT * FROM anggota ORDER BY nama")->fetchAll(PDO::FETCH_ASSOC);  
$daftarBukuTersedia = $pdo->query("SELECT * FROM buku WHERE stok > 0 ORDER BY judul")->fetchAll(PDO::
```

- `SELECT * FROM buku WHERE stok > 0` — inilah kode yang mewujudkan aturan bisnis yang sudah dirancang sejak [dokumentasi jobsheet-04 §5.5](#):
 - "Buku stok habis tidak boleh dipilih di form peminjaman."* Klausula `WHERE stok > 0` memastikan **hanya** buku yang stoknya tersedia yang muncul di dropdown — buku dengan stok `0` tidak akan pernah menjadi pilihan sama sekali.

```
<?php if (empty($daftarAnggota)): ?>  
    <p class="flash flash-error">Belum ada data anggota. Tambahkan anggota terlebih dahulu.</p>  
<?php elseif (empty($daftarBukuTersedia)): ?>  
    <p class="flash flash-error">Tidak ada buku dengan stok tersedia saat ini.</p>  
<?php else: ?>  
    <form>...</form>  
<?php endif; ?>
```

Ini pola percabangan **3 arah** (`if / elseif / else`) — belum pernah dipakai persis seperti ini sebelumnya di dokumentasi ini. Kalau tidak ada anggota sama sekali, atau tidak ada buku

tersedia sama sekali, form peminjaman **tidak ditampilkan** — hanya pesan error yang relevan. Form baru muncul kalau **kedua** syarat terpenuhi.

3.2 Menampilkan Pilihan yang Informatif

```
<option value="<?php echo $anggota['id']; ?>">
    <?php echo e($anggota['nama']); ?> (<?php echo e($anggota['no_anggota']); ?>)
</option>
```

Perhatikan `value` opsi ini berisi `id` (angka, dipakai server untuk mengetahui pilihan mana yang dikirim), sementara **teks yang tampil** ke pengguna menggabungkan nama **dan** nomor anggota — ingat** konsep beda `value` vs teks tampilan `<option>` dari [dokumentasi jobsheet-01 §4.4](#). Ini penting secara praktis: kalau ada dua anggota bernama sama persis ("Budi Santoso" misalnya), menampilkan nomor anggotanya sekaligus membantu Petugas memilih **orang yang benar**. Pola serupa (menyertakan stok di teks pilihan buku) juga dipakai untuk dropdown buku. Ingat `e()` dari [dokumentasi jobsheet-11 §2](#) dipakai membungkus setiap data teks yang berasal dari database.

3.3 proses_tambah.php: Kerangka Besarnya

```
require __DIR__ . '/../includes/auth.php';
require __DIR__ . '/../includes/csrf.php';
require __DIR__ . '/../includes/koneksi.php';

csrf_verify();

$anggotaId = $_POST['anggota_id'] ?? '';
$bukuId = $_POST['buku_id'] ?? '';

if ($anggotaId === '' || $bukuId === '') {
    $_SESSION['flash'] = ['type' => 'error', 'pesan' => 'Anggota dan buku wajib dipilih.'];
    header('Location: tambah.php');
    exit;
}

try {
    $pdo->beginTransaction();
    // ...
    $pdo->commit();
    // redirect sukses
} catch (Exception $e) {
    $pdo->rollBack();
}
```

```
// redirect gagal  
}
```

Kerangka luar file ini sudah sangat kamu kenal: `guard auth.php` ([dokumentasi jobsheet-10 §4](#)), `csrf_verify()` ([dokumentasi jobsheet-11 §3.7](#)), validasi wajib-isi ([dokumentasi jobsheet-07 §4.3](#)). Yang benar-benar baru adalah blok `try { ... beginTransaction ... commit } catch { ... rollBack }` — dibahas detail mulai [§3.4](#).

3.4 Transaction: `beginTransaction`, `commit`, `rollBack`

```
try {  
    $pdo->beginTransaction();  
  
    // ... beberapa perintah SQL ...  
  
    $pdo->commit();  
    $_SESSION['flash'] = ['type' => 'success', 'pesan' => 'Peminjaman berhasil dicatat.'];  
    header('Location: ../index.php');  
    exit;  
} catch (Exception $e) {  
    $pdo->rollBack();  
    $_SESSION['flash'] = ['type' => 'error', 'pesan' => 'Gagal mencatat peminjaman: ' . $e->getMessage()];  
    header('Location: tambah.php');  
    exit;  
}
```

- `$pdo->beginTransaction()`; — menandai **awal** sekelompok perintah SQL yang harus berhasil/gagal bersama-sama (ingat konsepnya dari [bab 1 §1.3](#)). Setelah baris ini, **semua** perintah `INSERT / UPDATE` berikutnya **belum benar-benar permanen** di database — mereka baru "dicoba," menunggu konfirmasi akhir.
- `$pdo->commit()`; — **mengunci** seluruh perubahan yang terjadi sejak `beginTransaction()` menjadi **permanen** di database sekaligus. Dipanggil **hanya** kalau seluruh perintah di dalamnya berhasil tanpa error.
- `$pdo->rollBack()`; — **membatalkan total** seluruh perubahan yang terjadi sejak `beginTransaction()`, seolah-olah **tidak pernah** terjadi sama sekali** — dipanggil di blok `catch`, tepat ketika ada** yang gagal di tengah proses.
- Pola `try / catch` di sini persis sama konsepnya dengan [dokumentasi jobsheet-06 §1.6](#), hanya sekarang dipakai untuk **transaction** SQL, bukan `fetch()`.

3.5 Mencegah Race Condition: `SELECT ... FOR UPDATE`

```

$cek = $pdo->prepare("SELECT stok FROM buku WHERE id = :id FOR UPDATE");
$cek->execute(['id' => $bukuId]);
$buku = $cek->fetch(PDO::FETCH_ASSOC);

if (!$buku || $buku['stok'] < 1) {
    throw new Exception('Stok buku tidak tersedia.');
```

- `FOR UPDATE` — klausa SQL yang **mengunci** baris yang diambil query ini, mencegah transaction **lain** membaca/mengubah baris yang sama **sampai** transaction ini selesai (`commit / rollback`). Ingat skenario race condition dari [bab 1 §1.4](#): kalau dua petugas mencoba meminjamkan buku stok-1 yang sama secara bersamaan, `FOR UPDATE` memastikan petugas **kedua** harus **menunggu** sampai transaction petugas **pertama** selesai — begitu menunggu selesai, petugas kedua akan membaca stok yang **sudah** diperbarui (kemungkinan besar sudah 0), sehingga pengecekan `$buku['stok'] < 1` di bawahnya akan **menolak** peminjaman kedua itu dengan benar, bukan malah membuat stok jadi negatif.
- `if (!$buku || $buku['stok'] < 1) { throw new Exception(...); }` — `throw new Exception(...)` adalah cara PHP "melempar" error secara manual (ingat konsep serupa `throw new Error(...)` di JavaScript dari [dokumentasi jobsheet-06 §4.5](#)). Melempar `Exception` di sini **langsung melompat** ke blok `catch` ([§3.4](#)), yang akan memanggil `rollback()` — membatalkan transaction ini **sebelum** sempat mengubah data apa pun, meski proses sudah sampai di tengah jalan.

3.6 Mencatat Peminjaman & Mengurangi Stok

```

$insert = $pdo->prepare(
    "INSERT INTO peminjaman (buku_id, anggota_id, tanggal_pinjam, status)
    VALUES (:buku_id, :anggota_id, CURRENT_DATE, 'dipinjam')";
);
$insert->execute(['buku_id' => $bukuId, 'anggota_id' => $anggotaId]);

$update = $pdo->prepare("UPDATE buku SET stok = stok - 1 WHERE id = :id");
$update->execute(['id' => $bukuId]);

$pdo->commit();
```

- `INSERT` baris baru ke `peminjaman` — pola yang sudah kamu kuasai sejak [dokumentasi jobsheet-08 §5](#). Perhatikan `CURRENT_DATE` ditulis **langsung** di query (bukan parameter) — meski kolom `tanggal_pinjam` sudah punya `DEFAULT CURRENT_DATE` ([bab 2 §2.3](#)), menuliskannya eksplisit di sini membuat maksud kode lebih jelas dibaca, meski secara teknis bisa juga dihilangkan dan mengandalkan nilai default.

- `UPDATE buku SET stok = stok - 1 WHERE id = :id` — perhatikan `stok = stok - 1`: nilai baru dihitung **berdasarkan nilai** lama**nya sendiri (ambil stok saat ini, kurangi 1), bukan menuliskan** angka tetap. Pola ini aman dipakai di sini **karena** sudah dilindungi `FOR UPDATE` di §3.5 — tidak ada transaction lain yang bisa mengubah nilai `stok` di tengah-tengah antara saat dibaca dan saat diperbarui.
- `$pdo->commit();` — dipanggil **hanya** setelah **kedua** perintah (`INSERT` dan `UPDATE`) berhasil dieksekusi tanpa error — inilah momen kedua perubahan itu **benar-benar** menjadi permanen di database, sekaligus, sesuai prinsip transaction dari [bab 1 §1.3](#).

Lanjut ke: [Pengembalian Buku](#)

4. Pengembalian Buku

Modul ini adalah **kebalikan** logis dari Peminjaman Baru ([bab 3](#)) — memakai pola transaction yang sama, hanya arah perubahannya dibalik.

4.1 peminjaman/kembali.php: Daftar Transaksi Aktif

```
$sqlDasar = "SELECT p.id, b.judul, a.nama, p.tanggal_pinjam
FROM peminjaman p
JOIN buku b ON b.id = p.buku_id
JOIN anggota a ON a.id = p.anggota_id
WHERE p.status = 'dipinjam';"
```

Query ini memakai `JOIN` — konsep yang dibahas mendalam di [bab 5](#), karena `riwayat.php` memakainya dengan cara yang lebih mudah dijelaskan langkah demi langkah. Inti singkatnya di sini: query ini menggabungkan data dari **3 tabel** sekaligus (`peminjaman`, `buku`, `anggota`) supaya bisa menampilkan **judul buku** dan **nama anggota** secara langsung, padahal tabel `peminjaman` sendiri hanya menyimpan `buku_id`/`anggota_id` berupa angka (ingat dari [bab 2](#)). `WHERE p.status = 'dipinjam'` memastikan **hanya** transaksi yang belum dikembalikan yang muncul di halaman ini.

Sisa halaman ini — kolom pencarian (`method="get"`, `ILIKE`), tombol "Kembalikan" dibungkus `<form method="post">` dengan `csrf_field()` dan `<input type="hidden" name="id">` — adalah pola yang **identik** dengan yang sudah kamu kenal sejak [dokumentasi jobsheet-09](#) dan [dokumentasi jobsheet-11](#).

4.2 proses_kembali.php: Transaction Kebalikan

```

try {
    $pdo->beginTransaction();

    $stmt = $pdo->prepare("SELECT buku_id, status FROM peminjaman WHERE id = :id FOR UPDATE");
    $stmt->execute(['id' => $id]);
    $trx = $stmt->fetch(PDO::FETCH_ASSOC);

    if (!$trx || $trx['status'] !== 'dipinjam') {
        throw new Exception('Transaksi tidak ditemukan atau sudah dikembalikan.');
```

```

    }

    $updatePeminjaman = $pdo->prepare(
        "UPDATE peminjaman SET status = 'dikembalikan', tanggal_kembali = CURRENT_DATE WHERE id = :id"
    );
    $updatePeminjaman->execute(['id' => $id]);

    $updateBuku = $pdo->prepare("UPDATE buku SET stok = stok + 1 WHERE id = :buku_id");
    $updateBuku->execute(['buku_id' => $trx['buku_id']]);

    $pdo->commit();
    $_SESSION['flash'] = ['type' => 'success', 'pesan' => 'Buku berhasil dikembalikan.'];
} catch (Exception $e) {
    $pdo->rollBack();
    $_SESSION['flash'] = ['type' => 'error', 'pesan' => 'Gagal memproses pengembalian: ' . $e->getMessage()];
}

```

Bandingkan langkah demi langkah dengan `proses_tambah.php` yang sudah kamu bedah di [bab 3](#):

	Peminjaman Baru	Pengembalian
Baris yang dikunci <code>FOR UPDATE</code>	Baris buku (cek stok)	Baris peminjaman (cek status)
Syarat kegagalan	<code>!\$buku \</code>	<code>\</code>
Perubahan pada tabel peminjaman	<code>INSERT baris baru, status = 'dipinjam'</code>	<code>UPDATE status = 'dikembalikan', isi tanggal_kembali</code>
Perubahan pada tabel buku	<code>stok = stok - 1</code>	<code>stok = stok + 1</code>

- `FOR UPDATE` **di sini mengunci baris** `peminjaman`, bukan `buku` — masuk akal, karena race condition yang perlu dicegah di sini berbeda: bayangkan **dua** klik tombol "Kembalikan"

untuk transaksi yang **sama**, hampir bersamaan (misalnya karena pengguna klik dua kali tidak sengaja) — tanpa penguncian, kedua proses bisa sama-sama lolos pengecekan `status === 'dipinjam'` dan **sama-sama** menambah stok buku (`stok + 1` dua kali), padahal pengembalian sungguhan hanya terjadi sekali.

- `$trx['status'] !== 'dipinjam'` — pengecekan ini penting: kalau transaksi yang sama entah bagaimana **sudah** berstatus `'dikembalikan'` (misalnya dari percobaan klik ganda tadi, atau transaksi yang memang tidak valid), proses ini **menolak** melanjutkan — mencegah stok bertambah dua kali untuk **satu** pengembalian buku yang sama.
- `UPDATE peminjaman SET status = 'dikembalikan', tanggal_kembali = CURRENT_DATE` — mengubah **dua kolom sekaligus** dalam satu perintah `UPDATE` (ingat sintaks `SET kolom1 = ..., kolom2 = ...` dari [dokumentasi jobsheet-09 §2.6](#)) — status berubah **dan** tanggal kembali terisi otomatis dengan tanggal hari ini, dalam satu langkah.

4.3 Kenapa Guard Method Masih Diperiksa Padahal Sudah Ada CSRF?

```
if ($_SERVER['REQUEST_METHOD'] !== 'POST') {  
    header('Location: kembali.php');  
    exit;  
}  
  
csrf_verify();
```

Ingat dari [dokumentasi jobsheet-09 §3.3](#), pemeriksaan `$_SERVER['REQUEST_METHOD']` mencegah operasi ini terpicu lewat `GET` (tautan/crawler). Meski token CSRF ([dokumentasi jobsheet-11 §3](#)) sudah menutup celah pemalsuan permintaan `POST` dari situs lain, kedua lapisan ini **melengkapi**, bukan menggantikan satu sama lain — pemeriksaan method mencegah kasus lebih sederhana (klik tautan biasa), sementara token CSRF mencegah kasus yang lebih canggih (form palsu yang juga memakai `POST`). Prinsip melapisi beberapa pertahanan sekaligus seperti ini disebut **defense in depth** — jangan bergantung pada satu lapisan proteksi saja.

Lanjut ke: [Riwayat Peminjaman & JOIN](#)

5. Riwayat Peminjaman & JOIN

Halaman paling sederhana secara alur (hanya membaca data, tidak ada transaction), tapi memperkenalkan konsep SQL penting: `JOIN`.

5.1 Apa itu JOIN?

Ingat dari [bab 1 §1.1](#), tabel `peminjaman` hanya menyimpan **angka** (`buku_id`, `anggota_id`) — bukan judul buku atau nama anggota secara langsung. Kalau kamu hanya menjalankan `SELECT * FROM peminjaman`, hasilnya berupa deretan angka yang tidak berarti apa-apa bagi pengguna (siapa yang mau melihat "buku_id: 3, anggota_id: 7"?).

`JOIN` adalah perintah SQL yang **menggabungkan** baris dari dua tabel (atau lebih) berdasarkan kecocokan kolom tertentu — biasanya foreign key dengan primary key yang dirujuknya ([dokumentasi jobsheet-12 §2.2](#)). Hasilnya: satu baris gabungan yang berisi kolom dari **kedua** tabel sekaligus.

5.2 Kode Query Lengkap

```
$stmt = $pdo->prepare(
    "SELECT b.judul, p.tanggal_pinjam, p.tanggal_kembali, p.status
    FROM peminjaman p
    JOIN buku b ON b.id = p.buku_id
    WHERE p.anggota_id = :id
    ORDER BY p.tanggal_pinjam DESC"
);
```

5.3 Alias Tabel: p dan b

```
FROM peminjaman p
JOIN buku b ON b.id = p.buku_id
```

- `peminjaman p` dan `buku b` — memberi tabel sebuah **alias** (nama pendek sementara): `peminjaman` disingkat `p`, `buku` disingkat `b`. Ini murni untuk **mempersingkat penulisan** query — tanpa alias, setiap penyebutan kolom harus menulis nama tabel lengkap (`peminjaman.tanggal_pinjam` alih-alih `p.tanggal_pinjam`), yang membuat query panjang menjadi sulit dibaca, apalagi kalau melibatkan banyak tabel sekaligus (ingat `kembali.php` di [bab 4 §4.1](#) yang menggabungkan 3 tabel: `p`, `b`, `a`).

5.4 Klausa ON: Syarat Penggabungan

```
JOIN buku b ON b.id = p.buku_id
```

ON b.id = p.buku_id adalah **syarat** bagaimana baris dari kedua tabel dicocokkan: "gabungkan baris buku dengan baris peminjaman **kalau** id di tabel buku sama dengan buku_id di tabel peminjaman." Inilah wujud konkret dari relasi foreign key yang sudah dibahas di [bab 2 §2.2](#) — JOIN ... ON "menerjemahkan" hubungan antar tabel itu menjadi data gabungan yang sesungguhnya bisa ditampilkan.

5.5 Memilih Kolom dari Kedua Tabel

```
SELECT b.judul, p.tanggal_pinjam, p.tanggal_kembali, p.status
```

Perhatikan daftar kolom yang diambil **bercampur** dari kedua tabel: b.judul (dari tabel buku), sementara p.tanggal_pinjam, p.tanggal_kembali, p.status (dari tabel peminjaman). Inilah manfaat JOIN yang sesungguhnya: satu baris hasil query ini terlihat seperti "satu tabel gabungan" yang berisi kolom relevan dari **kedua** sumber data, meski di database aslinya tersimpan sebagai dua tabel terpisah.

5.6 Menyaring Berdasarkan Anggota Terpilih

```
$anggotaId = $_GET['anggota_id'] ?? '';  
// ...  
if ($anggotaId !== '') {  
    // ...  
    $stmt->execute(['id' => $anggotaId]);  
    $riwayat = $stmt->fetchAll(PDO::FETCH_ASSOC);  
}
```

- WHERE p.anggota_id = :id — menyaring hasil JOIN supaya **hanya** menampilkan riwayat peminjaman milik **satu** anggota spesifik, sesuai wireframe yang sudah kamu bedah sejak [dokumentasi jobsheet-04 §2.3](#):
 - "Riwayat Peminjaman — Siti Aminah"*
- \$_GET['anggota_id'] — nomor anggota yang mau dilihat riwayatnya diambil dari URL (ingat konsep ini dari [dokumentasi jobsheet-09 §2.2](#)), dikirim lewat form method="get" yang berisi dropdown pilihan anggota.
- Kalau \$anggotaId masih kosong (halaman baru dibuka, belum memilih anggota apa pun), query JOIN ini **sama sekali tidak dijalankan** — \$riwayat tetap berupa array kosong yang sudah diinisialisasi di awal file, dan halaman hanya menampilkan form pemilihan anggota tanpa tabel riwayat.

5.7 Mempertahankan Pilihan Dropdown: Perbandingan String

```
<option value="<?php echo $anggota['id']; ?>" <?php echo (string) $anggotaId === (string) $anggota['
```

- Ingat pola `selected` dari [dokumentasi jobsheet-09 §2.4](#) — menandai opsi yang **sedang dipilih** supaya tetap terlihat terpilih setelah halaman submit ulang (mirip `value="..."` yang mempertahankan isi kotak pencarian, ingat dari [dokumentasi jobsheet-09 §5.8](#)).
- `(string) $anggotaId === (string) $anggota['id']` — kedua nilai **sengaja** di-cast ke `(string)` (ingat konsep type casting dari [dokumentasi jobsheet-07 §4.5](#)) sebelum dibandingkan dengan `===` (perbandingan **ketat**, yang memeriksa tipe data **dan** nilai sekaligus). `$anggotaId` dari `$_GET` **selalu** berupa string (ingat semua data dari `$_GET` / `$_POST` berbentuk teks apa adanya, dari [dokumentasi jobsheet-08 §5.4](#)), sementara `$anggota['id']` dari hasil `SELECT` biasanya sudah berupa **integer**. Tanpa `(string)` di kedua sisi, `===` akan **selalu** bernilai `false` (karena tipe data `"3"` dan `3` dianggap berbeda oleh `===`), meski nilainya "sama" secara makna — mengubah keduanya jadi string dulu memastikan perbandingan berjalan sesuai maksudnya.

5.8 Menampilkan Status yang Ramah Dibaca

```
<td><?php echo $r['status'] === 'dipinjam' ? 'Dipinjam' : 'Selesai'; ?></td>
```

Ingat ternary operator dari [dokumentasi jobsheet-05 §5.4](#) — kolom `status` di database menyimpan nilai teknis (`'dipinjam'` atau `'dikembalikan'`, ingat dari [bab 2 §2.4](#)), tapi ke pengguna ditampilkan label yang lebih ramah dibaca: "Dipinjam" atau "Selesai" — persis seperti yang dirancang di wireframe Riwayat Peminjaman sejak jobsheet-04.

Lanjut ke: [Rangkuman & Latihan Lanjutan](#)

6. Rangkuman & Latihan Lanjutan

6.1 Rangkuman Keseluruhan Jobsheet 12

Bagian	Konsep yang Dipelajari
Konsep Dasar Integrasi & Transaction	Kenapa butuh tabel relasi, foreign key, transaction, race condition
Skema peminjaman	REFERENCES, tipe DATE, DEFAULT CURRENT_DATE, relasi many-to-many
Peminjaman Baru	beginTransaction / commit / rollBack, SELECT ... FOR UPDATE
Pengembalian	Transaction kebalikan, defense in depth (method + CSRF)
Riwayat & JOIN	JOIN ... ON, alias tabel, perbandingan string (string) === (string)

6.2 Konsep Inti yang Perlu Diingat

1. **Tabel relasi (seperti peminjaman) menjembatani dua entitas** lewat foreign key, memungkinkan satu buku dipinjam banyak anggota dari waktu ke waktu, dan sebaliknya ([bab 1 §1.1-1.2](#)).
2. **Transaction memastikan beberapa perintah SQL berhasil/gagal bersama-sama**** — mencegah data setengah-jadi kalau salah satu** langkah gagal di tengah proses ([bab 3 §3.4](#)).
3. `SELECT ... FOR UPDATE` **mencegah race condition** dengan mengunci baris yang sedang diperiksa/diubah sampai transaction selesai — penting kapan pun ada kemungkinan dua proses mengubah data yang sama secara bersamaan ([bab 3 §3.5](#)).
4. `JOIN` **menggabungkan data dari beberapa tabel** menjadi satu hasil query, berdasarkan hubungan foreign key/primary key ([bab 5](#)).
5. **Defense in depth** — melapisi beberapa pertahanan sekaligus (pemeriksaan method HTTP dan token CSRF dan guard login) lebih aman daripada mengandalkan satu lapisan saja ([bab 4 §4.3](#)).

6.3 Cara Mencoba Sendiri

Ikuti alur pengujian end-to-end dari [README.md](#) jobsheet ini secara berurutan:

1. Jalankan skema tambahan dan server:

```
psql -d simpus_mini -f sql/03_peminjaman.sql
php -S localhost:8000
```

(atau lewat Laragon — lihat [README.md](#) jobsheet ini untuk opsi vhost-nya)

2. **Registrasi** petugas baru → **Login**.
3. **Tambah Buku** (kalau belum ada) & **Tambah Anggota** (kalau belum ada).
4. Klik **Peminjaman Baru**, pilih anggota dan buku, simpan — cek **Daftar Buku**, amati stok buku itu **berkurang 1**.
5. Kembali ke **Beranda**, amati kartu "Sedang Dipinjam" **bertambah** (ingat dari [README.md](#) jobsheet ini, angka ini kini dihitung sungguhan dari `COUNT(*) FROM peminjaman WHERE status = 'dipinjam'`, bukan lagi statis 0 sejak jobsheet-01).
6. Klik **Pengembalian**, cari transaksi yang baru dibuat, klik "Kembalikan" — cek stok buku **bertambah kembali**, dan transaksi **hilang** dari daftar aktif.
7. Klik **Riwayat**, pilih anggota yang sama — transaksi tadi **muncul**, berstatus "**Selesai**".
8. **Logout**, coba akses `/peminjaman/tambah.php` langsung — amati diarahkan ke Login (ingat `guard auth.php` dari [dokumentasi jobsheet-10 §4](#) berlaku juga di modul baru ini).

6.4 Ide Latihan Tambahan (Opsional)

1. **Terapkan validasi bisnis yang belum ada** — sesuai catatan di [README.md](#) jobsheet ini: anggota dengan peminjaman **terlambat lebih dari 14 hari** tidak boleh meminjam buku baru. Petunjuk: hitung selisih `CURRENT_DATE` dengan `tanggal_pinjam` pada transaksi aktif anggota tersebut sebelum mengizinkan `INSERT` baru di `proses_tambah.php`.
2. **Tambah kolom "jatuh tempo"** — misalnya `tanggal_jatuh_tempo DATE` yang otomatis diisi 14 hari setelah `tanggal_pinjam` (petunjuk: cari fungsi tanggal PostgreSQL seperti `tanggal_pinjam + INTERVAL '14 days'`), lalu tampilkan di `kembali.php` untuk membantu Petugas melihat transaksi mana yang sudah lewat jatuh tempo.
3. **Uji race condition secara manual** — buka dua tab browser berbeda (atau dua sesi terpisah), coba pinjamkan buku dengan stok tersisa 1 dari **kedua** tab hampir bersamaan. Amati hanya **satu** yang berhasil, yang lain mendapat pesan "Stok buku tidak tersedia." (bukti `FOR UPDATE` dari [bab 3 §3.5](#) bekerja).
4. **Tambah JOIN tambahan di** `kembali.php` / `riwayat.php` — misalnya sertakan juga kolom `no_hp` anggota (dari tabel `anggota`) di daftar transaksi aktif, sebagai latihan menambah kolom ke query `JOIN` yang sudah ada.

Kalau ada bagian yang masih membingungkan, terutama soal transaction dan `FOR UPDATE` di [bab 3](#), coba praktikkan langsung latihan uji race condition di [§6.4 poin 3](#) — melihat sendiri bagaimana permintaan kedua "menunggu" dan akhirnya ditolak adalah cara paling nyata untuk memahami kenapa penguncian baris ini penting, melampaui sekadar membaca penjelasan teoretisnya.

Checklist Keamanan — Jobsheet 11

Audit menyeluruh terhadap kode Jobsheet 7-10, dengan bukti before/after.

#	Kerentanan	Ditemukan di	Sebelum	Sesudah (perbaikan)
1	SQL Injection	Semua query di <code>buku/</code> , <code>anggota/</code> , <code>auth/</code>	Sejak Jobsheet 8 sudah memakai prepared statement PDO (<code>:parameter</code>)	Diaudit ulang, sudah aman — tidak ada satupun query yang menyisipkan <code>\$_POST</code> / <code>\$_GET</code> langsung ke string SQL. Diuji input <code>' OR '1'='1</code> di form login → tidak berhasil bypass.
2	XSS (Cross-Site Scripting)	<code>buku/list.php</code> , <code>buku/edit.php</code> , <code>anggota/list.php</code> , <code>anggota/edit.php</code> , <code>includes/header.php</code> (nama petugas)	Output <code>judul</code> , <code>pengarang</code> , <code>nama</code> , <code>alamat</code> , <code>no_hp</code> , dan nilai pencarian (<code>q</code>) dicetak langsung tanpa escaping	Dibungkus fungsi <code>e()</code> (<code>includes/helpers.php</code> , <code>htmlspecialchars</code> dengan <code>ENT_QUOTES</code>). Diuji simpan judul <code>buku <script>alert(1)</script></code> → tampil sebagai teks biasa, bukan dieksekusi.
3	CSRF (Cross-Site Request Forgery)	Form Tambah/Edit/Hapus Buku & Anggota, Login, Register	Form POST tidak memiliki token verifikasi — bisa dipicu dari situs lain	Ditambah <code>includes/csrf.php</code> (<code>csrf_field()</code> + <code>csrf_verify()</code>), token disimpan di <code>\$_SESSION['csrf_token']</code> , diverifikasi di setiap <code>proses_*.php</code> dan <code>hapus.php</code> sebelum query dijalankan.
4	Validasi & Sanitasi Input	<code>proses_tambah.php</code> , <code>proses_edit.php</code> (<code>buku</code> & <code>anggota</code>)	Sudah ada validasi tipe (<code>is_numeric</code>)	Diaudit ulang, tetap dipertahankan — ditambah cast eksplisit (<code>int</code>) pada <code>id</code> di

#	Kerentanan	Ditemukan di	Sebelum	Sesudah (perbaikan)
			dan wajib-isi sejak Jobsheet 7-9	form Edit untuk mencegah nilai non-numerik masuk sebagai hidden input.
5	Session Fixation	auth/proses_login.php	Session ID tidak diperbarui setelah login	<code>session_regenerate_id(true)</code> dipanggil tepat setelah <code>password_verify()</code> berhasil.

Catatan Implementasi

- Guard `includes/auth.php` selalu dijalankan **sebelum** `includes/csrf.php` di halaman proses — memastikan pengguna yang belum login tidak bisa memicu pengecekan CSRF sama sekali (langsung di-redirect ke login).
- Fungsi `csrf_verify()` mengembalikan HTTP 403 dan menghentikan eksekusi (`die()`) bila token tidak cocok atau tidak ada — diverifikasi manual dengan mengirim POST tanpa `csrf_token` ke `proses_tambah.php` (setelah login).